Facultatea de Matematică şi Informatică

Domeniul Informatică

Didactica Informaticii

Problema 31

Rotaru Ștefan, Grupa 234, stefan.rotaru@s.unibuc.ro

Anul Universitar 2025-2026

Curpins

0. Enunțul problemei: 2
1. Breviar teoretic: 3
2. Cum abordăm problema? 4
3. Dificultatea problemei 6
4. Idei pentru cod 7
5. Implementare C++ 8
6. Implementare Python 10
7. Complexitatea algoritmului 12
8. Barem de notare propus: 13

0. Enunțul problemei:

Într-o sală trebuie programate N spectacole. Pentru fiecare spectacol se cunoaște intervalul [s, f) de desfășurare. Să se determine numărul maxim de spectacole ce pot fi programate astfel încât ele să nu se suprapună.

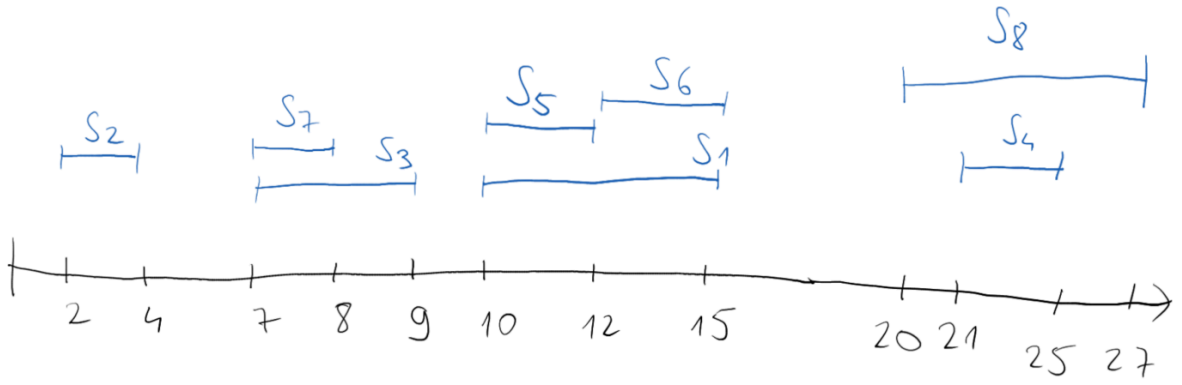
Exemplu:
Pentru n = 8 și [10, 15) [2, 4) [7, 9) [21, 25) [10, 12) [12, 15) [7, 8) [20, 27) se va afișa: Spectacolele alese sunt 2 7 5 6 4
Numărul de spectacole este 5

1. Breviar teoretic:

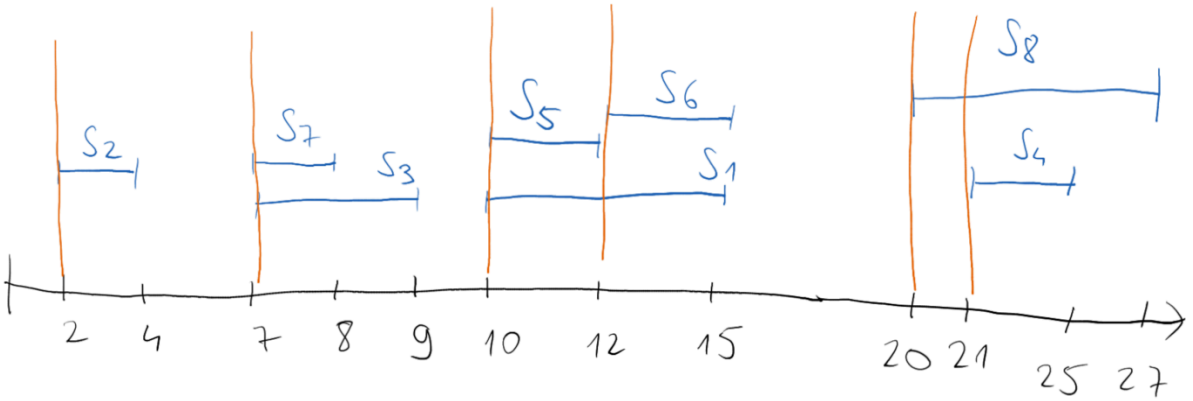
Această problemă este cunoscută ca și “Problema spectacolelor” și este o problemă clasică de Greedy. Această metodă de programare presupune luarea unei decizii optime la fiecare pas, cunoscut și ca optim local, în speranța că acesta va duce la un optim global. Deoarece spectacolele nu sunt ordonate încă, ar trebui să le sortăm, deci este necesară și cunoașterea unui algoritm de selecție, fie sortarea prin selecție, fie metoda bulelor. Pentru soluția C++ este recomandată pentru ușurarea lucrului cunoștința structurilor, pentru ca să putem grupa cele trei date de intrare cu care vine un spectacol (id, start, final), iar pentru soluția Python, din aceleași motive, familiaritatea cu conceptul de liste de liste sau liste de tupluri, pentru a salva pentru fiecare spectacol (id, start, final).

2. Cum abordăm problema?

Pentru a ne da seama cum ar trebui sortate, ar trebui să ne dăm seama după care din cei doi termeni din input ai fiecărui spectacol (start sau final) să sortăm după. Mai jos este axa timpului pe care am așezat fiecare spectacol.

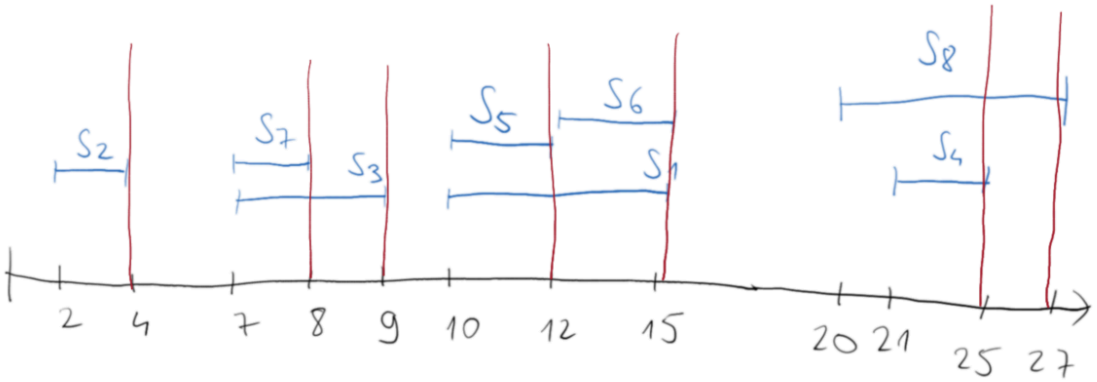


Varianta 1: Sortăm după timpul de începere (start)



În acest fel, noua ordine a spectacolelor ar fi S2, S3, S7, S1, S5, S6, S8, S4. De aici se poate observa o problemă: există spectacole care încep în același moment (S3 și S7 încep la momentul 7, S1 și S5 încep la momentul 10). Deși pentru S3 și S7 nu există o problemă, între S1 și S5 există, întrucât dacă ar fi fost luat S5 în loc de S1, ar mai fi avut loc și S6, deci soluția nu este optimă.

Varianta 2: Sortăm după timpul de terminare (final)



În acest fel, noua ordine a spectacolelor este S2, S7, S3, S5, S1, S6, S4, S8. De data aceasta, nu mai avem probleme, căci vor fi luate S5 și S6 în fața lui S1, aceasta fiind soluția optimă.

3. Dificultatea problemei

Fiind o problemă de tip Greedy, partea cea mai grea este identificarea acesteia ca fiind una de problemă Greedy și cum găsim soluția optimă. Pentru cineva neinițiat, soluția ar părea să fie sortarea după timpul de începere a spectacolelor sau după durata spectacolelor și să se ducă să implementeze această soluție, fără să realizeze că aceasta nu este soluția optimă. Ca la orice algoritm de tip Greedy, trebuie găsită mai întâi strategia optimă (de cele mai multe ori pe lucrul cu inputul, cum am făcut pe schemele de mai sus), iar apoi codul este doar o sortare, o parcurgere și o verificare. O confuzie a enunțului ar putea fi datorată din interpretarea nefastă a intervalelor, unde s-ar putea crede că dacă un spectacol se termină la 14, nu ar putea începe un spectacol care începe la 14.

4. Idei pentru cod

- Citire date de intrare, stocare într-un vector/ într-o listă elemente de tipul (id, start, final)
- Sortare vector/listă după *final*
- Parcurgere vector/listă și verificare dacă final[i] <= start[j] (j>i) și salvare a ultimului spectacol
- Incrementare a contorului și afișarea spectacolului (id)
- Afișare contor

5. Implementare C++

```
#include <iostream>
using namespace std;


// Structura pentru a grupa datele unui spectacol
struct Spectacol {
    int id;
    int start;
    int final; };


int main() {
    int n;
    Spectacol S[100];
    cin >> n;
    for (int i = 0; i < n; i++) {
        S[i].id = i + 1; // Salvăm numărul spectacolului (1, 2, 3...)
        cin >> S[i].start >> S[i].final;
    }


    // Sortare crescătoare după ora de final (Metoda Bulelor)
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (S[j].final > S[j+1].final) {
                // Interschimbăm structurile complet
                Spectacol aux = S[j];
                S[j] = S[j+1];
                S[j+1] = aux;
            }
        }
    }


    // Pasul Greedy: Alegem primul spectacol (cel care se termină cel mai repede)
    cout << "Spectacolele alese sunt ";
    int ultimul_f = S[0].final;
```

```
    cout << S[0].id << " ";
    int count = 1;


    // Verificăm restul spectacolelor
    for (int i = 1; i < n; i++) {
        // Dacă spectacolul curent începe după sau exact când se termină ultimul ales
        if (S[i].start >= ultimul_f) {
            cout << S[i].id << " ";
            ultimul_f = S[i].final; // Actualizăm ora de final curentă
            count++;
        }
    }


    cout << "\nNumarul de spectacole este " << count << "\n";


    return 0;
}
```

6. Implementare Python

```
# Citirea datelor
n = int(input("Introduceți numărul de spectacole: "))
spectacole = []


for i in range(n):
# Citim o linie, o despărțim după spații și convertim în întregi
    date = input().split()
    start = int(date[0])
    final = int(date[1])
    # Adăugăm un tuplu de forma (ID, start, final)
    spectacole.append((i + 1, start, final))


# Căutăm la fiecare pas minimul din porțiunea nesortată a listei
for i in range(len(spectacole) - 1):
    min_idx = i
    for j in range(i + 1, len(spectacole)):
        # Comparăm după ora de final (indexul 2 din tuplu: (id, start, final))
        if spectacole[j][2] < spectacole[min_idx][2]:
            min_idx = j

    # Dacă am găsit un spectacol care se termină mai devreme, interschimbăm (swap)
    if min_idx != i:
        spectacole[i], spectacole[min_idx] = spectacole[min_idx], spectacole[i]




# Pasul Greedy: Alegem primul spectacol sortat
alese = [spectacole[0]]


# Verificăm restul spectacolelor
for i in range(1, len(spectacole)):
    # spectacole[i][1] este ora de start a spectacolului analizat
    # alese[-1][2] este ora de final a ultimului spectacol adăugat
    if spectacole[i][1] >= alese[-1][2]:
```

```
        alese.append(spectacole[i])


# Afișarea datelor
print("Spectacolele alese sunt:", end=" ")
for sp in alese:
    print(sp[0], end=" ")


print(f"\nNumărul de spectacole este {len(alese)}") # Se putea face și cum am făcut la C++, dar am ales să folosesc proprietățile limbajului
```

7. Complexitatea algoritmului

Acest algoritm are complexitatea dependentă de algoritmul de sortare folosit (în cazul nostru $O(n^2)$), dar folosind algoritmi de sortare mai buni (quicksort, mergesort, funcția predefinită în Python sorted(), metoda predefinită în Python .sort()), complexitatea ar fi $O(n * \log(n))$.

8. Barem de notare propus:

- Citirea și stocarea datelor: 1p (utilizarea corectă a structurilor/listelor, alternativ se poate lucra cu 3 vectori/liste, unul pentru id, unul pentru start și unul pentru final)
- Criteriul de sortare Greedy: 3p (alegerea sortării după ora de terminare, alternativ o altă rezolvare care nu este Greedy, dar se vor pierde punctele pentru compexitate, iar baremul de mai jos nu s-ar mai fi aplicat. Se va da 1p pentru sortare după ora de începere și 0p pentru sortare după durată)
- Algoritmul de sortare: 2p (orice algoritm de sortare. La nivel de liceu sunt permiși și algoritmii $O(n^2)$ (depinde de cum va fi scrisă programa pentru la anul pentru Python, metodele și funcțiile predefinite pot sau nu să fie permise))
- Logica selecției: 2p (verificarea condiției start[j] >= final[i] (scrisă evident conform limbajelor în care este implementat), 1p pentru omiterea egalității)
- Afișarea rezultatelor în formatul așteptat: 1p (Respectă formatul din exemplu)
- Complexitatea $O(n^2)$: 1p